

PhantomLog — Guía de React para Juniors

Parte 1: Fundamentos, Componentes y `useState`

Esta guía está escrita específicamente para el proyecto PhantomLog. Todos los ejemplos son código real de tu aplicación.

Índice de las 3 partes

Parte	Contenido
Parte 1 (este archivo)	Qué es React, JSX, Componentes, <code>useState</code>
Parte 2	<code>useEffect</code> , <code>useContext</code> , hooks de React Router
Parte 3	<code>useCallback</code> , <code>useMemo</code> , <code>useRef</code> , patrones avanzados

¿Qué es React y por qué lo usamos?

React es una **librería de JavaScript** que permite construir interfaces de usuario dividiendo la pantalla en piezas reutilizables llamadas **componentes**.

El principio fundamental es: **en lugar de manipular el HTML directamente, describes cómo quieres que se vea la pantalla y React se encarga de actualizarla cuando los datos cambian.**

Sin React (vanilla JS):

```
// Buscas el elemento y lo cambias manualmente
document.getElementById('contador').innerText = 5
```

Con React:

```
// Describes el estado, React actualiza el HTML solo
const [contador, setContador] = useState(0)
// En el JSX: <div>{contador}</div>
// Cuando llamas setContador(5), la pantalla se actualiza sola
```

¿Qué es JSX?

JSX es la sintaxis que ves en tus archivos `.jsx`. Parece HTML pero es JavaScript.

Ejemplo de tu `Login.jsx`:

```
return (  
  <div className="vh100 flex-center relative overflow-hidden">  
    <form onSubmit={handleSubmit} className="horror-form">  
      <h1>IDENTIFICARSE</h1>  
      <input  
        type="email"  
        value={email}  
        onChange={e => setEmail(e.target.value)}  
      />  
      <button disabled={loading}>  
        {loading ? 'AUTENTICANDO...' : 'INICIAR SESIÓN'}  
      </button>  
    </form>  
  </div>  
)
```

Reglas importantes de JSX:

- `class` → `className` (porque `class` es una palabra reservada de JS)
- Los atributos de eventos son camelCase: `onclick` → `onClick`, `onchange` → `onChange`
- Todo expresión JavaScript va entre llaves `{}`: `{loading ? 'A' : 'B'}`
- Un componente solo puede devolver **un elemento raíz**. Si necesitas devolver dos cosas sin un `<div>` extra, usa `<>...</>` (Fragment)

Componentes: La Base de Todo

Un componente en React es simplemente una **función que devuelve JSX**. En PhantomLog, cada página y cada pieza de UI es un componente.

Tipos de componentes en tu proyecto:

1. Componentes de Página (Pages)

Son las pantallas completas. React Router decide cuál mostrar según la URL.

```
// src/pages/Cart.jsx  
export default function Cart() {  
  // lógica aquí  
  return (  
    <div className="page-container">  
      { /* HTML de la página del carrito */ }  
    </div>  
  )  
}
```

2. Componentes Reutilizables (Components)

Son piezas que se usan en múltiples páginas.

```
// src/components/ShimmerImage.jsx
const ShimmerImage = ({ src, alt, objectFit = "cover" }) => {
  // lógica de carga con shimmer
  return (
    <div className="shimmer-container">
      <img src={src} alt={alt} />
    </div>
  )
}
export default ShimmerImage
```

Fíjate en `{ src, alt, objectFit = "cover" }`: eso son las **props** (propiedades). Son los parámetros que le pasas al componente desde fuera:

```
// Usándolo en Forums.jsx
<ShimmerImage
  src={f.image_url}
  alt={f.title}
  objectFit="cover" // <-- esta es una prop
/>
```

useState — El Hook de Estado Local

`useState` es el hook más fundamental. Permite que un componente **recuerde** información entre renderizados.

Sintaxis

```
const [valor, setValor] = useState(valorInicial)
```

- `valor` → El dato actual
- `setValor` → La función para cambiarlo (NUNCA cambies `valor` directamente)
- `valorInicial` → Lo que tiene al principio (`null`, `false`, `[]`, `''`, `0`)

¿Por qué no puedo hacer `valor = 'nuevo'`?

Porque React no sabría que algo cambió. Al llamar `setValor('nuevo')`, React:

1. Guarda el nuevo valor
2. Vuelve a ejecutar el componente (re-renderiza)
3. El JSX muestra el nuevo valor automáticamente

Ejemplo 1: Estado de Formulario en `Login.jsx`

```
export default function Login() {
  const [email, setEmail] = useState('') // string vacío
  const [password, setPassword] = useState('') // string vacío
  const [loading, setLoading] = useState(false) // booleano

  const handleSubmit = async (e) => {
    e.preventDefault() // evita que el form recargue la página
    setLoading(true) // activa el spinner

    try {
      await login(email, password)
    } catch (err) {
      // manejo de error
    } finally {
      setLoading(false) // desactiva el spinner pase lo que pase
    }
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        value={email}
        onChange={e => setEmail(e.target.value)} // actualiza con cada tecla
      />
      <button disabled={loading}>
        {loading ? 'AUTENTICANDO...' : 'INICIAR SESIÓN'}
      </button>
    </form>
  )
}
```

Flujo:

1. Usuario escribe en el input → dispara `onChange`
2. `onChange` llama `setEmail(e.target.value)`
3. React actualiza el estado y re-renderiza
4. El `value={email}` del input ahora muestra el texto actualizado

Esto se llama **Controlled Component**: el estado de React controla siempre lo que muestra el input.

Ejemplo 2: Estado Complejo en `Cart.jsx`

```
const [cartData, setCartData] = useState(null)
const [loading, setLoading] = useState(true)
const [updatingState, setUpdatingState] = useState({}) // Objeto vacío
```

`updatingState` es un **objeto** que actúa como mapa. Almacena qué productos están siendo modificados en este momento:

```
// Cuando el usuario pulsa "eliminar" en el producto con id=3:
setUpdatingState(prev => ({ ...prev, [productId]: 'rem' }))
// updatingState ahora es: { 3: 'rem' }

// En el JSX, desactivamos el botón si ese producto está en proceso:
<button disabled={updatingState[item.product.id]}>ELIMINAR</button>
```

El patrón `prev => ({ ...prev, nueva_clave: valor })` se usa para actualizar **parte** de un objeto sin perder el resto. Es equivalente a:

```
const copia = { ...updatingState } // copia todo lo que había
copia[productId] = 'rem'           // añade/modifica solo esta clave
setUpdatingState(copia)            // guarda la copia modificada
```

Ejemplo 3: Estado con Array en `DataProvider.jsx`

```
const [products, setProducts] = useState([])
const [forums, setForums] = useState([])
```

Cuando llegan los datos de la API:

```
const res = await getProducts(params)
const data = res.data.data || res.data
setProducts(Array.isArray(data) ? data : []) // guarda el array o [] si falla
```

Renderizado Condicional

En JSX puedes mostrar u ocultar partes según condiciones:

Patrón 1: Operador `&&` (muestra si es verdadero)

```
// En ForumDetail.jsx: solo muestra los botones de editar si el usuario es el
creador
{user && String(user.id) === String(forum?.user_id) && (
  <div className="flex-center gap-10">
    <button onClick={() => setShowForumModal(true)}>EDITAR FORO</button>
    <button onClick={handleDeleteForum}>ELIMINAR FORO</button>
  </div>
)}
```

Patrón 2: Ternario `? :`

```
// En Cart.jsx: texto del botón según si está cargando
<button type="submit" disabled={isSubmitting}>
  {isSubmitting ? 'PROCESANDO...' : `CONSECRAR PAGO: ${total}€`}
</button>
```

Patrón 3: Retorno temprano

```
// En ProductDetail.jsx: si no se encuentra el producto, muestra NotFound
if (notFound) return <NotFound />
if (loading) return <div>CARGANDO...</div>
if (!product) return null // nada si aún no hay datos pero tampoco hay error
```

Renderizado de Listas con `.map()`

Para mostrar arrays en JSX se usa `.map()`. **Cada elemento necesita una `key` única.**

```
// En Products.jsx
{products.map(p => (
  <div key={p.id} className="horror-card">
    <h3>{p.title}</h3>
    <span>{Number(p.price).toFixed(2)}€</span>
  </div>
))}
```

¿Por qué la `key`? React usa las keys para identificar qué elementos han cambiado cuando la lista se actualiza. Sin key, React recrearía todos los elementos desde cero en cada re-render, lo cual es ineficiente y puede causar bugs visuales.

Regla de oro: Usa siempre el `id` real del dato como key. Nunca uses el índice del array (`i`) como key en listas que pueden cambiar de orden.

Eventos y Handlers

Los manejadores de eventos en React son funciones que se pasan como props:

```
// Inline (para lógica simple)
<button onClick={() => navigate('/products')}>VOLVER</button>

// Función declarada (para lógica compleja)
const handleDelete = async (id) => {
  if (!window.confirm('¿Seguro?')) return
  await deleteItem(id)
```

```
}  
<button onClick={() => handleDelete(item.id)}>BORRAR</button>
```

¿Por qué () => handleDelete(item.id) y no solo handleDelete?

Porque necesitamos pasarle el argumento `item.id`. Si escribimos solo `{handleDelete}`, se ejecutaría al renderizar sin argumentos. Envolviendo en una función flecha controlamos cuándo y con qué argumentos se llama.

Resumen del Capítulo

Concepto	Cuándo usarlo
<code>useState(null)</code>	Para datos que aún no han llegado
<code>useState(false)</code>	Para flags de carga o visibilidad
<code>useState([])</code>	Para listas de datos
<code>useState({})</code>	Para gestionar múltiples estados con un mapa
Operador <code>&&</code>	Para mostrar algo solo si una condición es verdadera
Ternario <code>? :</code>	Para elegir entre dos opciones
Retorno temprano	Para páginas de carga, error o "no encontrado"

→ Continúa en la Parte 2: `useEffect`, `useContext` y React Router

PhantomLog — Guía de React para Juniors

Parte 2: `useEffect`, `useContext` y React Router

Prerequisito: haber leído la Parte 1 (Fundamentos y `useState`)

`useEffect` — Efectos Secundarios

`useEffect` es el hook para ejecutar código que **no forma parte del renderizado** en sí mismo: llamadas a APIs, timers, suscripciones a eventos del navegador, etc.

Se llama "efecto secundario" (side effect) porque es código que afecta a algo externo al componente.

Anatomía de `useEffect`

```
useEffect(() => {  
  // 1. Código del efecto (se ejecuta)  
  fetchData()  
})
```

```
// 2. Función de limpieza (OPCIONAL, se ejecuta antes del próximo efecto o al
desmontar)
return () => {
  cleanup()
}
}, [dep1, dep2]) // 3. Array de dependencias (controla CUÁNDO se ejecuta)
```

El Array de Dependencias: La clave de todo

El array `[]` del final es lo más importante de entender.

Sintaxis	Cuándo se ejecuta
Sin array	En CADA re-render (¡peligroso!)
<code>[]</code> vacío	Solo una vez, al montar el componente
<code>[dep1, dep2]</code>	Cada vez que <code>dep1</code> o <code>dep2</code> cambien

Caso 1: `[]` vacío — "Carga inicial"

En `AuthContext.jsx`, comprobamos si el usuario tiene sesión guardada **solo una vez** al iniciar la app:

```
// src/context/AuthContext.jsx
useEffect(() => {
  if (localStorage.getItem('token')) {
    me() // Llamada a la API: "¿quién soy?"
      .then(res => setUser(res.data)) // Si hay token válido, guarda el usuario
      .catch(() => localStorage.removeItem('token')) // Si el token expiró,
    bórralo
      .finally(() => setLoading(false)) // En cualquier caso, deja de cargar
  } else {
    setLoading(false) // Sin token, no hay que esperar nada
  }
}, []) // <-- [] vacío = solo se ejecuta una vez al montar
```

¿Qué pasaría si no pusiera `[]`? Se ejecutaría en cada re-render, haciendo una llamada a la API infinitamente.

Caso 2: Dependencias — "Reaccionar a cambios"

En `Login.jsx`, redirigimos al usuario si ya tiene sesión:

```
// src/pages/Login.jsx
useEffect(() => {
  if (user) {
    navigate('/dashboard') // Si el usuario existe, mándalo al dashboard
  }
}, [user])
```



```
}
}, [user, navigate]) // <-- Se ejecuta cada vez que `user` cambie
```

Flujo:

1. Página carga → `user` es `null` → el efecto se ejecuta pero `if(user)` es falso, nada pasa
2. Usuario hace login → `user` pasa a tener datos → el efecto se ejecuta de nuevo → redirección

También en `ProductDetail.jsx`, recargamos el producto si cambia el `id` de la URL:

```
// src/pages/ProductDetail.jsx
useEffect(() => {
  getProduct(id)
    .then(res => setProduct(res.data))
    .catch(() => setNotFound(true))
    .finally(() => setLoading(false))
}, [id]) // <-- Si el usuario navega de /products/3 a /products/7, recarga
```

Caso 3: Cleanup — "Cancelar efectos anteriores"

El caso más importante del proyecto: el **debouncing** en la búsqueda de productos.

```
// src/pages/Products.jsx
useEffect(() => {
  const params = {
    search: globalSearch,
    page: currentPage,
    per_page: 9,
    sort: activeFilters.sort
  }

  if (globalSearch !== '') {
    // Esperamos 400ms antes de llamar a la API
    const handler = setTimeout(() => {
      refreshProducts(params)
    }, 400)

    // CLEANUP: Si el usuario escribe otra letra antes de 400ms,
    // cancelamos el timer anterior
    return () => clearTimeout(handler)
  } else {
    // Sin búsqueda, cargamos inmediatamente
    refreshProducts(params)
  }
}, [globalSearch, currentPage, activeFilters, refreshProducts])
```

¿Qué resuelve esto?

Sin debouncing: el usuario escribe "fantasma" → 8 llamadas a la API (una por letra) Con debouncing: el usuario escribe "fantasma" → 1 llamada (400ms después de la última letra)

¿Cómo funciona el cleanup?

1. Usuario escribe "f" → se crea timer de 400ms
2. Usuario escribe "fa" (antes de 400ms) → el cleanup cancela el timer de "f" → se crea nuevo timer para "fa"
3. Usuario termina de escribir → timer llega a 400ms → se llama a la API con "fantasma"

Caso 4: `useEffect` en `ForumDetail.jsx` — Carga múltiple

```
// src/pages/ForumDetail.jsx
useEffect(() => {
  fetchForum() // Carga el foro
  fetchReports() // Carga los reportes del foro
}, [id]) // Cuando cambia el id del foro en la URL, recarga todo
```

Este es un patrón muy común: cargar todos los datos relacionados de una página cuando el componente monta o cuando el ID cambia.

Errores comunes con `useEffect`

Error 1: Bucle infinito

```
// ❌ MAL: fetchData cambia en cada render, triggering el efecto infinitamente
useEffect(() => {
  fetchData()
}, [fetchData]) // si fetchData no está en useCallback, es una nueva función cada render

// ✅ BIEN: fetchData está en useCallback, su referencia es estable
const fetchData = useCallback(async () => { ... }, [])
useEffect(() => {
  fetchData()
}, [fetchData])
```

Error 2: Dependencia faltante

```
// ❌ MAL: usamos `id` pero no lo ponemos en las dependencias
useEffect(() => {
  getProduct(id) // si `id` cambia, este efecto NO se vuelve a ejecutar
}, [])

// ✅ BIEN
useEffect(() => {
```

```
    getProduct(id)  
  }, [id])
```

Error 3: Operaciones asíncronas directas

```
// ❌ MAL: useEffect no puede ser async directamente  
useEffect(async () => {  
  const data = await fetchData()  
}, [])  
  
// ✅ BIEN: define la función async dentro y llámala  
useEffect(() => {  
  const load = async () => {  
    const data = await fetchData()  
  }  
  load()  
}, [])
```

useContext — Estado Global

El problema que resuelve: Prop Drilling

Imagina que `user` está en el componente raíz y lo necesitas en una tarjeta de forum profundamente anidada:

```
App  
├── Layout  
│   ├── ForumDetail  
│   │   ├── ReportCard  
│   │   │   └── CommentBox ← necesita saber si el usuario está logueado
```

Sin contexto, tendrías que pasar `user` como prop en cada nivel. Con contexto, cualquier componente puede acceder directamente a él.

Cómo está implementado en PhantomLog

El proyecto tiene **4 contextos**:

Contexto	Qué provee	Hook de acceso
AuthContext	user, login, logout, updateUser	useAuth()
DataProvider	products, forums, expeditions, refresh...	useData()
CartContext	cartCount, fetchGlobalCart, setCartCount	useCart()
ToastContext	addToast	useToast()

Anatomía de un Contexto: `AuthContext.jsx`

```
// 1. Crear el contexto (es el "canal" de comunicación)
const AuthContext = createContext(null)

// 2. El Provider: envuelve a los hijos y les provee los datos
export function AuthProvider({ children }) {
  const [user, setUser] = useState(null)
  const [loading, setLoading] = useState(true)

  // Al montar, comprueba si hay token guardado
  useEffect(() => {
    if (localStorage.getItem('token')) {
      me().then(res => setUser(res.data))
        .finally(() => setLoading(false))
    } else {
      setLoading(false)
    }
  }, [])

  const login = async (email, password) => {
    const { data } = await apiLogin({ email, password })
    localStorage.setItem('token', data.token) // Guarda el token en el navegador
    setUser(data.user) // Actualiza el estado global
  }

  const logout = async () => {
    await apiLogout()
    localStorage.removeItem('token') // Borra el token del navegador
    setUser(null) // Limpia el usuario del estado
  }

  return (
    // 3. Pasa los datos a través del Provider
    <AuthContext.Provider value={{ user, login, logout, updateUser, loading }}>
      {loading ? <Loader /> : children}
    </AuthContext.Provider>
  )
}

// 4. Hook personalizado para consumir el contexto fácilmente
export const useAuth = () => useContext(AuthContext)
```

Consumir el Contexto

Desde cualquier componente hijo del `AuthProvider`:

```
// src/pages/Login.jsx
const { user, login } = useAuth()
```

```
// src/pages/Forums.jsx
const { user } = useAuth()

// src/pages/Profile.jsx
const { user, updateUser, logout } = useAuth()
```

El DataProvider: Contexto de Datos Globales

`DataProvider.jsx` es el contexto más complejo. Centraliza **todos los datos de la app**:

```
// src/context/DataProvider.jsx
export function DataProvider({ children }) {
  // Un estado por cada tipo de dato
  const [products, setProducts] = useState([])
  const [forums, setForums] = useState([])
  const [expeditions, setExpeditions] = useState([])
  // ...

  // Estados de carga por separado
  const [loadingProducts, setLoadingProducts] = useState(false)
  const [loadingForums, setLoadingForums] = useState(false)
  // ...

  // Funciones para recargar datos (con useCallback para optimizar)
  const refreshForums = useCallback(async (params = {}) => {
    const requestId = ++requestRefs.current.forums // Anti race-condition
    setLoadingForums(true)
    try {
      const res = await getForums(params)
      if (requestId !== requestRefs.current.forums) return // Descarta peticiones
      const data = res.data.data || res.data
      setForums(Array.isArray(data) ? data : [])
    } catch (error) {
      console.error('Error fetching forums:', error)
    } finally {
      setLoadingForums(false)
    }
  }, [])

  // Al hacer login, carga todos los datos de una vez
  const { user } = useAuth()
  useEffect(() => {
    if (user) {
      refreshAll() // Llama a todos los refresh en paralelo
    }
  }, [user, refreshAll])

  // El valor del contexto está memoizado para evitar re-renders innecesarios
```

```
const value = useMemo(() => ({
  products, loadingProducts, productsPagination, refreshProducts,
  forums, loadingForums, forumsPagination, refreshForums,
  // ...
}), [products, forums, /* ... */])

return (
  <DataContext.Provider value={value}>
    {children}
  </DataContext.Provider>
)
```

¿Por qué `useMemo` en el valor del contexto?

Sin `useMemo`, cada vez que cualquier estado del `DataProvider` cambia (por ejemplo, `loadingProducts`), se crea un nuevo objeto para el `value`. React interpretaría que el contexto cambió y **re-renderizaría todos los componentes** que lo consumen. Con `useMemo`, el objeto solo se recrea cuando las dependencias reales cambian.

React Router — Navegación entre Páginas

React Router te permite mostrar distintos componentes según la URL del navegador, sin recargar la página.

Hooks de React Router usados en PhantomLog

`useNavigate` — Navegar por código

```
const navigate = useNavigate()

// Ir a una página
navigate('/dashboard')
navigate('/forums')

// Ir atrás
navigate(-1)

// Navegar con un ID dinámico
navigate(`/forums/${forum.id}`)
navigate(`/success/${resp.data.id}`)
```

Ejemplo real en `Login.jsx`:

```
// Si el usuario ya está logueado, mándalo al dashboard automáticamente
useEffect(() => {
  if (user) {
    navigate('/dashboard')
  }
})
```

```
}  
}, [user, navigate])
```

Ejemplo real en `Checkout.jsx`:

```
// Tras completar el pago, espera 3 segundos y redirige  
setTimeout(() => navigate(`/success/${resp.data.id}`), 3000)
```

`useParams` — Leer parámetros de la URL

Cuando la ruta tiene un parámetro dinámico como `/forums/:id`, `useParams` te da acceso a ese valor.

```
// src/pages/ForumDetail.jsx  
const { id } = useParams()  
// Si la URL es /forums/12, entonces id = "12"  
  
// src/pages/ReportDetail.jsx  
const { id: forumId, reportId } = useParams()  
// Renombramos 'id' a 'forumId' para más claridad  
// URL: /forums/3/reports/7 → forumId = "3", reportId = "7"
```

Fíjate que el `id` viene siempre como **string**, por eso en el proyecto a veces verás `String(user.id) === String(f.user_id)` para comparar IDs sin problemas de tipo.

`Link` — Navegación declarativa

`Link` de React Router es como un `<a>` pero sin recargar la página.

```
// En lugar de <a href="/register">, usamos:  
<Link to="/register" className="fs-12 opacity-07">  
  ¿SIN CREDENCIALES? REGISTRAR NUEVA ALMA  
</Link>  
  
// En Expeditions.jsx, cada tarjeta es un Link  
<Link to={`/expeditions/${exp.id}`} className="horror-card column gap-15">  
  <h3>{exp.name.toUpperCase()}</h3>  
  { /* ... */ }  
</Link>
```

¿Cuándo usar `Link` vs `useNavigate`?

- `Link`: Para navegación declarativa visible en el HTML (como un enlace de menú o una tarjeta clickable)

- **useNavigate**: Para navegación programática (tras un submit de formulario, tras un temporizador, en respuesta a un evento)

Protección de Rutas

En **App.jsx** (o donde se definen las rutas), hay componentes que protegen ciertas páginas. Si el usuario no está logueado, lo redirigen al login:

```
// Ejemplo de patrón de ruta protegida
function PrivateRoute({ children }) {
  const { user } = useAuth()
  return user ? children : <Navigate to="/login" />
}

// Uso
<Route path="/cart" element={
  <PrivateRoute>
    <Cart />
  </PrivateRoute>
} />
```

Resumen del Capítulo

Hook / Concepto	Cuándo usarlo
<code>useEffect(fn, [])</code>	Una vez al montar el componente
<code>useEffect(fn, [dep])</code>	Cuando <code>dep</code> cambia
<code>return () => cleanup()</code>	Para cancelar timers o suscripciones
<code>createContext</code> + <code>Provider</code>	Para compartir datos globalmente
<code>useContext(MyContext)</code>	Para consumir ese contexto
<code>useNavigate</code>	Para redirigir por código
<code>useParams</code>	Para leer parámetros de la URL
<code><Link to=""></code>	Para enlaces en el HTML

→ Continúa en la Parte 3: **useCallback**, **useMemo**, **useRef** y Patrones Avanzados

PhantomLog — Guía de React para Juniors

Parte 3: **useCallback**, **useMemo**, **useRef** y Patrones Avanzados

Prerequisito: haber leído las Partes 1 y 2.

Introducción a la Optimización

Cuando React renderiza un componente, ejecuta **toda la función** del componente de nuevo. Esto incluye recalcular variables, recrear funciones y recalcular objetos. La mayoría de las veces es suficientemente rápido, pero cuando:

- Pasas funciones como dependencias de `useEffect`
- Tienes cálculos costosos (filtrar miles de elementos)
- Compartes datos por Contexto con muchos consumidores

...necesitas los hooks de optimización.

`useCallback` — Estabilizar Referencias de Funciones

El Problema

En JavaScript, dos funciones con el mismo código son **objetos distintos**:

```
const fn1 = () => console.log('hola')
const fn2 = () => console.log('hola')
fn1 === fn2 // false ← son objetos distintos en memoria
```

Esto causa un problema: si defines una función dentro de un componente y la usas en `useEffect`, React ve una nueva función en cada render y ejecuta el efecto infinitamente.

La Solución: `useCallback`

`useCallback` "memoriza" una función y devuelve **siempre la misma referencia** a menos que cambien sus dependencias.

```
const miFuncion = useCallback(() => {
  // código
}, [dep1, dep2]) // se recrea SOLO si dep1 o dep2 cambian
```

En PhantomLog: `DataProvider.jsx`

```
// src/context/DataProvider.jsx

// SIN useCallback: se crea una nueva función en cada render del DataProvider
// Todos los useEffect que dependan de refreshForums ejecutarían infinitamente
const refreshForums = async (params = {}) => { ... } // ❌ problema

// CON useCallback: la misma función siempre (el array [] significa "nunca
recrear")
const refreshForums = useCallback(async (params = {}) => {
  setLoadingForums(true)
```

```

    try {
      const res = await getForums(params)
      setForums(res.data.data || res.data)
    } catch (error) {
      console.error('Error fetching forums:', error)
    } finally {
      setLoadingForums(false)
    }
  }, []) // [] = esta función nunca necesita recrearse

```

¿Por qué [] vacío aquí? Porque `refreshForums` solo usa `setLoadingForums` y `setForums`, que son funciones estables de `useState` (React garantiza que no cambian).

Cuando las dependencias no están vacías

Si la función usa valores externos que pueden cambiar:

```

// Ejemplo hipotético: una función que filtra según un parámetro del componente
const buscarForos = useCallback((termino) => {
  return forums.filter(f => f.title.includes(termino))
}, [forums]) // se recrea cuando `forums` cambie

```

useMemo — Memorizar Valores Calculados

El Problema

Los cálculos costosos se repiten en cada render, incluso si los datos de entrada no cambiaron.

La Solución: `useMemo`

`useMemo` ejecuta una función y **guarda el resultado**. Solo recalcula cuando las dependencias cambian.

```

const resultado = useMemo(() => {
  return calculo_costoso(datos)
}, [datos]) // recalcula solo cuando `datos` cambie

```

En PhantomLog: Filtrado en `Expeditions.jsx`

Sin `useMemo`, cada vez que el usuario mueve el ratón (o cualquier re-render), React recorre **toda la lista de expediciones** para filtrarlas, aunque el texto de búsqueda no haya cambiado:

```

// SIN useMemo: se filtra en CADA render ❌
const filteredExpeditions = expeditions.filter(exp =>
  exp.name.toLowerCase().includes(search.toLowerCase())
)

```

```
// CON useMemo: solo se filtra cuando `expeditions` o `search` cambian ✅
const filteredExpeditions = useMemo(() => {
  return expeditions.filter(exp =>
    exp.name.toLowerCase().includes(search.toLowerCase()) ||
    exp.location.toLowerCase().includes(search.toLowerCase())
  )
}, [expeditions, search]) // <-- dependencias

// La paginación también se memoiza porque depende del resultado del filtrado
const totalPages = useMemo(
  () => Math.ceil(filteredExpeditions.length / itemsPerPage),
  [filteredExpeditions]
)

const paginatedExpeditions = useMemo(
  () => filteredExpeditions.slice(startIndex, startIndex + itemsPerPage),
  [filteredExpeditions, startIndex]
)
```

En PhantomLog: El Valor del Contexto en `DataProvider.jsx`

Este es el uso más importante de `useMemo` en el proyecto:

```
// SIN useMemo: cada vez que loading cambia, se crea un nuevo objeto
// React lo interpreta como un nuevo contexto → re-renderiza TODA la app ❌
return (
  <DataContext.Provider value={{
    products, forums, expeditions, refreshAll, ...
  }}>
```

```
// CON useMemo: el objeto solo cambia cuando algo dentro realmente cambió ✅
const value = useMemo(() => ({
  products, loadingProducts, productsPagination, refreshProducts,
  forums, loadingForums, forumsPagination, refreshForums,
  invoices, loadingInvoices, invoicesPagination, refreshInvoices,
  phantoms, loadingPhantoms, refreshPhantoms,
  expeditions, loadingExpeditions, refreshExpeditions,
  globalSearch, setGlobalSearch,
  refreshAll
}), [
  products, loadingProducts, /* todas las dependencias */
])

return (
  <DataContext.Provider value={value}>
    {children}
  </DataContext.Provider>
)
```

```
</DataContext.Provider>  
)
```

Impacto real: Sin `useMemo`, al cargar los productos, se re-renderizarían también todos los foros, expediciones y el carrito. Con `useMemo`, solo se re-renderizan los componentes que realmente usan `products`.

`useMemo` vs `useCallback`

Hook	Para	Devuelve
<code>useMemo</code>	Memorizar un valor calculado	El resultado de la función
<code>useCallback</code>	Memorizar una función	La función misma

```
// useMemo: memoriza el RESULTADO  
const total = useMemo(() => items.reduce((acc, i) => acc + i.price, 0), [items])  
  
// useCallback: memoriza la FUNCIÓN en sí  
const calcularTotal = useCallback(() => {  
  return items.reduce((acc, i) => acc + i.price, 0)  
}, [items])
```

`useRef` — La Memoria Silenciosa

`useRef` crea una referencia mutable que **no provoca re-renders** cuando cambia.

```
const miRef = useRef(valorInicial)  
// Accedes/cambias el valor así:  
miRef.current = nuevoValor  
console.log(miRef.current) // lee el valor
```

¿Cuándo usar `useRef` en lugar de `useState`?

- `useState`: Cuando cambiar el valor debe **actualizar la pantalla**
- `useRef`: Cuando el valor es interno/técnico y **no necesita afectar la UI**

En PhantomLog: Anti Race Conditions en `DataProvider.jsx`

¿Qué es una Race Condition?

Imagina que el usuario escribe "f", luego rápidamente "fa". Se lanzan 2 peticiones a la API. La segunda ("fa") puede llegar antes que la primera ("f"). Si no hay protección, los datos de "f" sobrescriben a los de "fa" y la pantalla muestra resultados incorrectos.

```
// PROBLEMA SIN PROTECCIÓN:
// Petición 1: search="f" → tarda 500ms
// Petición 2: search="fa" → tarda 100ms
// Resultado: "fa" llega primero, luego llega "f" y sobrescribe... ✖

// SOLUCIÓN CON useRef:
const requestRefs = useRef({ products: 0, forums: 0, invoices: 0 })

const refreshForums = useCallback(async (params = {}) => {
  // Incrementa el contador de esta categoría y guarda ESTE número
  const requestId = ++requestRefs.current.forums

  setLoadingForums(true)
  try {
    const res = await getForums(params)

    // Comprueba si ESTA petición sigue siendo la más reciente
    if (requestId !== requestRefs.current.forums) {
      return // Descartamos este resultado, ya hay uno más nuevo ✔
    }

    setForums(res.data.data || res.data)
  } catch (error) {
    console.error(error)
  } finally {
    setLoadingForums(false)
  }
}, [])
```

¿Por qué `useRef` y no `useState`? Porque cambiar el contador de peticiones no debe re-renderizar el componente. Solo es un número interno de control. Con `useState`, actualizarlo causaría re-renders innecesarios.

`useRef` para Referencias al DOM

`useRef` también se usa para acceder directamente a elementos HTML:

```
// En ShimmerImage.jsx
const imgRef = useRef(null)

useEffect(() => {
  // Comprueba si la imagen ya estaba cargada antes de que el componente montara
  if (imgRef.current && imgRef.current.complete) {
    setLoaded(true)
  }
}, [src])

return (
```

```
<img ref={imgRef} src={src} onLoad={() => setLoaded(true)} />
)
```

`imgRef.current` apunta directamente al elemento `<img>` del DOM. Puedes leer `.complete` (si ya estaba cargada), `.width`, `.height`, etc.

Patrones Avanzados en PhantomLog

Patrón 1: Actualizaciones Optimistas

Usadas en `Cart.jsx` para una experiencia sin latencia. La idea es: actualiza la UI inmediatamente y luego confirma con el servidor.

```
const handleUpdate = async (productId, intent) => {
  // 0. Si es borrado, pide confirmación ANTES de tocar la UI
  if (intent === 'rem') {
    if (!window.confirm("¿CONFIRMAS QUE DESEAS PURGAR ESTA OFRENDA?")) {
      return // Usuario canceló → no hacemos nada
    }
  }

  // 1. Bloquea el botón de ese producto
  setUpdatingState(prev => ({ ...prev, [productId]: intent }))

  // 2. ACTUALIZACIÓN OPTIMISTA: cambia la UI inmediatamente
  setCartData(prev => {
    const newItems = prev.items.map(item => {
      if (item.product.id === productId) {
        if (intent === 'add') return { ...item, quantity: item.quantity + 1 }
        if (intent === 'sub') return { ...item, quantity: Math.max(0,
item.quantity - 1) }
      }
      return item
    }).filter(item => item.quantity > 0)

    if (intent === 'rem') {
      return { ...prev, items: prev.items.filter(i => i.product.id !== productId)
    }
    return { ...prev, items: newItems }
  })

  try {
    // 3. Llama a la API (en segundo plano)
    let res
    if (intent === 'add') res = await addToCart(productId, 1)
    if (intent === 'sub') res = await subtractCart(productId)
    if (intent === 'rem') res = await removeCart(productId)

    // 4. Confirma con los datos reales del servidor
```

```
    setCartData(res.data)
  } catch (e) {
    // 5. Si falló, restaura los datos reales (rollback)
    await fetchCart(true)
    addToast("Error en la sincronización.", "error")
  } finally {
    // 6. Siempre desbloquea el botón
    setUpdatingState(prev => {
      const next = { ...prev }
      delete next[productId]
      return next
    })
  }
}
```

Ventaja: El usuario ve el cambio en milisegundos, sin esperar la respuesta del servidor. **Riesgo:** Si el servidor falla, hay que revertir. Por eso el `catch` hace un `fetchCart(true)` (carga silenciosa que restaura los datos reales).

Patrón 2: Hooks Personalizados (Custom Hooks)

En el proyecto, los contextos se exponen como hooks personalizados:

```
// En AuthContext.jsx
export const useAuth = () => useContext(AuthContext)

// En DataProvider.jsx
export const useData = () => useContext(DataContext)

// En CartContext.jsx
export function useCart() {
  return useContext(CartContext)
}
```

Un hook personalizado es simplemente **una función que empieza por `use`** y puede llamar a otros hooks dentro.

Podrías crear tus propios hooks para extraer lógica repetida:

```
// Hook personalizado hipotético para manejar un formulario
function useForm(initialValues) {
  const [values, setValues] = useState(initialValues)
  const [loading, setLoading] = useState(false)

  const handleChange = useCallback((e) => {
    setValues(prev => ({ ...prev, [e.target.name]: e.target.value }))
  }, [])
}
```

```
    return { values, loading, setLoading, handleChange }  
  }
```

Patrón 3: Skeleton Loading con ShimmerImage

El componente `ShimmerImage` implementa una transición suave entre el estado de carga y el contenido real:


```
// src/components/ShimmerImage.jsx
const ShimmerImage = ({ src, alt, objectFit = "cover" }) => {
  const [loaded, setLoaded] = useState(false) // ¿está cargada?
  const [error, setError] = useState(false) // ¿falló la carga?
  const imgRef = useRef(null)

  useEffect(() => {
    // Caso especial: si la imagen ya estaba en caché del navegador,
    // `onLoad` no se dispara. Comprobamos manualmente.
    if (imgRef.current && imgRef.current.complete) {
      setLoaded(true)
    }
  }, [src])

  return (
    <div className="shimmer-container">
      {/* Shimmer: visible SOLO mientras carga */}
      {!loaded && !error && (
        <div className="shimmer-box" style={{ position: 'absolute', inset: 0 }} />
      )}

      {/* Error: visible si la imagen no se pudo cargar */}
      {error && (
        <div style={{ /* estilos de error */ }}>
          Imagen no disponible
        </div>
      )}

      {/* Imagen: siempre en el DOM pero invisible hasta que carga */}
      <img
        ref={imgRef}
        src={src}
        alt={alt}
        onLoad={() => setLoaded(true)} // Al cargar, hace fade in
        onError={() => setError(true)} // Al fallar, muestra mensaje de error
        style={{
          opacity: loaded ? 1 : 0, // Invisible hasta que cargue
          transition: 'opacity 0.5s ease-in-out' // Fade in suave
        }}
      />
    </div>
  )
}
```

Patrón 4: Estado de Carga con **finally**

Un patrón consistente en todo el proyecto para garantizar que **loading** siempre se desactiva:

```
const [loading, setLoading] = useState(false)

const handleSubmit = async () => {
  setLoading(true) // Activa siempre

  try {
    await operacionAsync()
    // Éxito
  } catch (error) {
    // Error
    addToast('Error', 'error')
  } finally {
    setLoading(false) // Se ejecuta SIEMPRE, tanto si hubo éxito como error
  }
}
```

El bloque `finally` garantiza que aunque haya un error inesperado, el botón no se quede bloqueado para siempre.

Árbol de Dependencias de PhantomLog

Entender el árbol del proyecto ayuda a saber qué componente puede acceder a qué contexto:

<code><AuthProvider></code>	→ provee <code>useAuth()</code>
<code><DataProvider></code>	→ provee <code>useData()</code> (necesita user de <code>AuthContext</code>)
<code><CartProvider></code>	→ provee <code>useCart()</code>
<code><ToastProvider></code>	→ provee <code>useToast()</code>
<code><Router></code>	
<code><Header /></code>	→ usa <code>useAuth()</code> , <code>useCart()</code> , <code>useData()</code>
<code><Routes></code>	
<code><Login /></code>	→ usa <code>useAuth()</code> , <code>useToast()</code>
<code><Forums /></code>	→ usa <code>useData()</code> , <code>useAuth()</code> , <code>useToast()</code>
<code><Cart /></code>	→ usa <code>useCart()</code> , <code>useToast()</code>
<code><Profile /></code>	→ usa <code>useAuth()</code> , <code>useToast()</code>
<code></Routes></code>	
<code><Footer /></code>	
<code></Router></code>	
<code></ToastProvider></code>	
<code></CartProvider></code>	
<code></DataProvider></code>	
<code></AuthProvider></code>	

Resumen General de Hooks

Hook	Propósito	Re-render al cambiar
<code>useState</code>	Guardar estado local	✅ Sí
<code>useEffect</code>	Efectos secundarios (APIs, timers)	No directamente
<code>useContext</code>	Consumir contexto global	✅ Sí (si el contexto cambia)
<code>useRef</code>	Referencia mutable sin re-render	❌ No
<code>useCallback</code>	Estabilizar referencia de función	❌ No (por sí solo)
<code>useMemo</code>	Memorizar valor calculado	❌ No (por sí solo)
<code>useNavigate</code>	Navegar entre rutas	❌ No
<code>useParams</code>	Leer parámetros de la URL	❌ No

Reglas de los Hooks (Obligatorias)

1. **Solo llama hooks en el nivel superior del componente.** Nunca dentro de `if`, `for` o funciones anidadas.
2. **Solo llama hooks desde componentes de React o hooks personalizados.** No desde funciones JS normales.
3. **Los hooks personalizados deben empezar por `use`** (convenio de React para identificarlos).

```
// ❌ MAL: hook dentro de condicional
if (condition) {
  const [state, setState] = useState(0) // Error!
}

// ✅ BIEN: hook siempre al nivel superior
const [state, setState] = useState(0)
if (condition) {
  setState(1) // Esto sí está bien
}
```

Guía de Referencia Rápida

¿Qué hook necesito?

```
¿Necesito guardar un dato que cambia y actualiza la pantalla?
→ useState

¿Necesito ejecutar código cuando el componente carga o cambia un dato?
→ useEffect

¿Necesito acceder a datos globales (usuario, productos, carrito)?
→ useAuth() / useData() / useCart() / useToast()
```

¿Necesito navegar a otra página por código?

→ `useNavigate`

¿Necesito leer el ID o parámetro de la URL?

→ `useParams`

¿Necesito estabilizar una función para usarla en `useEffect` o pasar a hijo?

→ `useCallback`

¿Necesito memorizar un cálculo costoso?

→ `useMemo`

¿Necesito un valor interno que no deba provocar re-renders?

→ `useRef`

Documentación generada para PhantomLog — Horror Minimalist Interface Versión del proyecto: Mayo 2026